UNIT – I

Java features – Java Platform – Java Fundamentals – Expressions, Operators, and Control Structures Classes and Objects, Constructors – Destructors - Packages and Interfaces – Internationalization.

Features of Java

There is given many features of java. The Java Features given below are simple and easy to understand.

1. Simple
2. Object-Oriented
3. Portable
4. Platform independent
5. Secured
6. Robust
7. Architecture neutral
8. Dynamic
9. Interpreted
10. High Performance
11. Multithreaded
12. Distributed

Simple

Java language is simple because:

- SNo need to remove unreferenced objects because there is Automatic Garbage Collection in java.
- syntax is based on C++
- Removed many confusing and rarely-used features such as pointers, operator overloading

Object-oriented

- Object-oriented means that organize our software as a combination of different types of objects that incorporates both data and behaviour.
- Basic concepts of OOPs are:
  - Object
  - Class
  - Inheritance
  - Polymorphism
  - Abstraction
  - Encapsulation

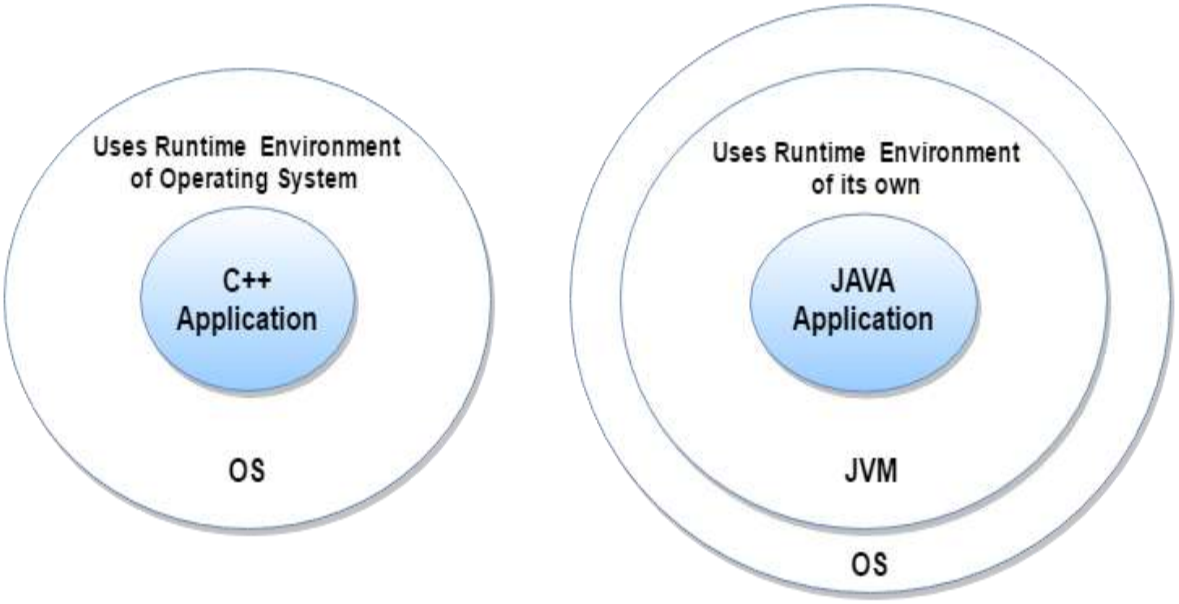
Platform Independent

- A platform is the hardware or software environment in which a program runs.
- Code can be run on multiple platforms e.g. Windows, Linux, Sun Solaris, Mac/OS etc.

Secured

Java is secured because:

- **No explicit pointer**
- **Java Programs run inside virtual machine sandbox**



- **Classloader:** adds security by separating the package for the classes of the local file system from those that are imported from network sources.
- **Bytecode Verifier:** checks the code fragments for illegal code that can violate access right to objects.
- **Security Manager:** determines what resources a class can access such as reading and writing to the local disk.

Robust

- Robust simply means strong.
- Java uses strong memory management. There are **lack of pointers that avoids security problem.**
- There is **automatic garbage collection** in java. There is **exception handling and type checking mechanism** in java.

Architecture-neutral

- There is **no implementation dependent features** e.g. size of primitive types is fixed.
- In C programming, **int data type** occupies **2 bytes of memory for 32-bit** architecture and **4 bytes of memory for 64-bit** architecture. But in java, it occupies **4 bytes of memory for both 32 and 64** bit architectures.

Portable

- We may carry the java bytecode to any platform.

High-performance

- Java is faster than traditional interpretation since **byte code is "close" to native code** still somewhat slower than a compiled language (e.g., C++)

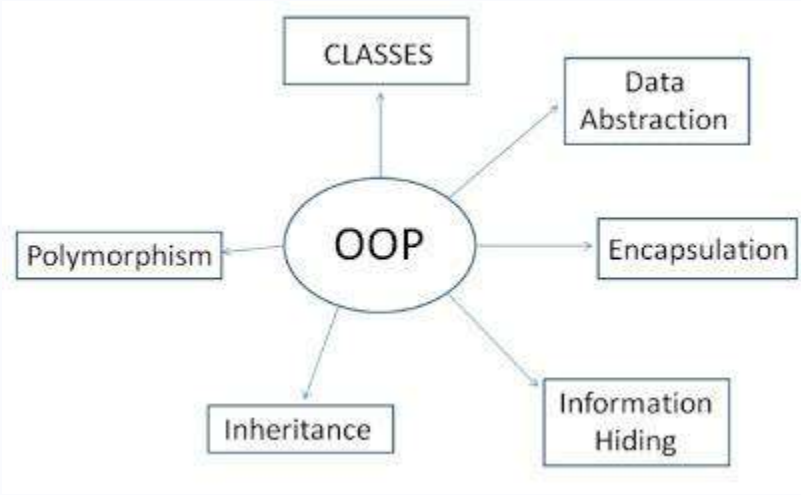
Distributed

We can create **distributed applications** in java**. RMI and EJB** are used for creating distributed applications. We may access files by calling the methods from any machine on the internet.

Multi-threaded

- A thread is like a separate program, executing concurrently. We can write Java programs that deal with **many tasks at once by defining multiple threads**.
- The main advantage of multi-threading is that **it doesn't occupy memory for each thread**. It **shares a common memory area.** Threads are important for multi-media, Web applications etc.

Features of Object Oriented Programming (OOP)



FEATURES OF OOP:

1. Object
2. Class
3. Data Hiding and Encapsulation
4. Dynamic Binding
5. Message Passing
6. Inheritance
7. Polymorphism

OBJECT: Object is a collection of number of entities. Objects take up space in the memory. Objects are instances of classes. When a program is executed , the objects interact by sending messages to one another. Each object contain data and code to manipulate the data. Objects can interact without having know details of each others data or code.

CLASS: Class is a collection of objects of similar type. Objects are variables of the type class. Once a class has been defined, we can create any number of objects belonging to that class. Eg: grapes bannans and orange are the member of class fruit.

Example:

Fruit orange;

In the above statement object mango is created which belong to the class fruit.

NOTE: Classes are user define data types.

DATA ABSTRACTION AND ENCAPSULATION:

Combining data and functions into a single unit called **class** and the process is known as **Encapsulation**.Data encapsulation is important feature of a class. Class contains both data and functions. Data is not accessible from the outside world and only those function which are present in the class can access the data. The insulation of the data from direct access by the program is called data hiding or information hiding. Hiding the complexity of proram is called **Abstraction** and only essential features are represented.In short we can say that internal working is hidden.

DYNAMIC BINDING: Refers to linking of function call with function defination is called binding and when it is take place at run time called dynamic binding.

MESSAGE PASSING: The process by which one object can interact with other object is called message passing.

POLYMORPHISM: A greek term means ability to take more than one form. An operation may exhibite different behaviours in different instances. The behaviour depends upon the types of data used in the operation.

Example:

- Method Overloading
- Method Overriding

Classes and Objects

Class

A class is a group of objects which have common properties. It is a template or blueprint from which objects are created. It is a logical entity. It can't be physical.

A class in Java can contain:

- **fields**
- **methods**
- **constructors**
- **blocks**
- **nested class and interface**

Syntax to declare a class:

```
class <class_name>{
   field;
   method;
}
```

Object

Object is an instance of a class. Class is a template or blueprint from which objects are created. So object is the instance(result) of a class.

Object Definitions:

- Object is *a real world entity*.
- Object is *a run time entity*.
- Object is *an entity which has state and behavior*.
- Object is *an instance of a class*.

Method

- A method is like function i.e. used to expose behavior of an object.

new keyword

- The new keyword is used to allocate memory at run time.
- All objects get memory in Heap memory area.

Object and Class Example: Rectangle

```
class Rectangle{
 int length;
 int width;
 void insert(int l, int w){
  length=l;
  width=w;
 }
 void calculateArea(){System.out.println(length*width);}
}

class TestRectangle1{
 public static void main(String args[]){
  Rectangle    r1=new Rectangle();
  Rectangle     r2=new Rectangle();
  r1.insert(11,5);
  r2.insert(3,15);
  r1.calculateArea();
  r2.calculateArea();
}
}
```

Output:

```
55
45
```

Constructors

• A constructor has the same name as the class.

• A class can have more than one constructor.

• A constructor can take zero, one, or more parameters.

• A constructor has no return value.

• A constructor is always called with the new operator.

Eg:-

```
import java.io.*;

class Student
{
	int marks;
	String name;
	public Student(int m,String s)
	{
		marks=m;
		name=s;
	}
	public void display()
	{
		System.out.println("Name: "+name+ "\nMark: "+marks);
	}
}

class EX
{
	public static void main(String args[])
	{
		Student st= new Student(100,"sachin");
		st.display();
	}
}
```

Output:

F:\ java>java EX

Name: sachin

Mark: 100

Rules for creating java constructor

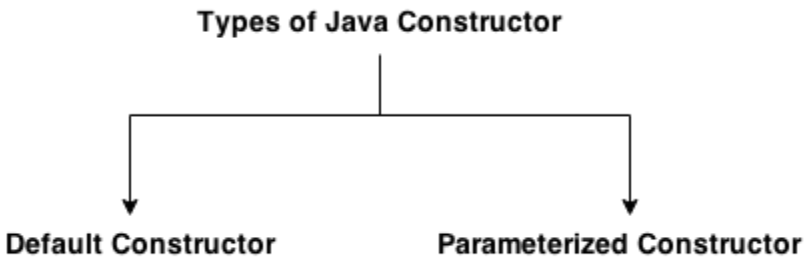
There are basically two rules defined for the constructor.

1. Constructor name must be same as its class name
2. Constructor must have no explicit return type

Types of java constructors

There are two types of constructors:

1. Default constructor (no-arg constructor)
2. Parameterized constructor



Java Default Constructor

A constructor that have no parameter is known as default constructor.

Syntax of default constructor:

```
<class_name>(){}
```

Example of default constructor

```
class Bike1{
Bike1(){System.out.println("Bike is created");}
public static void main(String args[]){
```

```
        Bike1 b=new Bike1();

        }

        }
```

Output:

```
Bike is created
```

Java parameterized constructor

A constructor that have parameters is known as parameterized constructor

```
class Student4{
   int id;
   String name;

   Student4(int i,String n){
   id = i;
   name = n;
   }
   void display(){System.out.println(id+" "+name);}

   public static void main(String args[]){
   Student4 s1 = new Student4(111,"Karan");
   Student4 s2 = new Student4(222,"Aryan");
   s1.display();
   s2.display();
  }
}
```

Output:

```
111 Karan
222 Aryan
```

Static Fields and Methods

Static Fields

A field as static, then there is only one such field per class. In contrast, each object has its own copy of all instance fields.

When a number of objects are created from the same class, each instance has its own copy of class variables. But this is not the case when it is declared as static **static**.

Eg:

```
import java.io.*;
class Demo
{
        static int i=20;
        public void display()
        {
                System.out.println (i);
        }
}

class EX
{
        public static void main(String args[])
        {
                Demo obj1,obj2;
                obj1=new Demo();
                obj2=new Demo();
                ++obj1.i;
                System.out.print ("x in obj1 :");
                obj1.display ();
                System.out.print ("x in obj2 :");
                obj2.display ();
        }
}
```

OUTPUT:

```
F:\java>java EX
x in obj1 :21
x in obj2 :21
```

Static Methods

- Static methods are methods that do not operate on objects.
- static methods can access the static fields in their class

static methods in two situations:

• When a method doesn't need to access the object state because all needed parameters are supplied as explicit parameters

• When a method only needs to access static fields of the class

Program:

```
import java.io.*;
class Student
{
static int marks;
static String name;

        public  void result(int m,String s)
        {
                marks=m;
                name=s;
        }

        public static void display()
        {
                System.out.println("Name: "+name+" \nMark: "+marks);
        }
}

class EX
{
        public static void main(String args[])
        {
                        Student st=new Student();
                        st.result(100,"sachin");
                        Student.display();
        }

}
```

Output:

```
F:\ java>java EX
Name: sachin
Mark: 100
```

Package

- A **java package** is a group of similar types of classes, interfaces and sub-packages.

Advantage of Java Package

- Java package is used to categorize the classes and interfaces so that they can be easily maintained.
- Java package provides access protection.
- Java package removes naming collision.

Simple example of java package

- The **package keyword** is used to create a package in java.

```
//save as Simple.java
package mypack;
public class Simple{
public static void main(String args[]){
System.out.println("Welcome to package");
}
}
```

Compile java package

Syntax

```
javac -d directory javafilename
```

example

```
javac -d . Simple.java
```

Run java package program

To Compile: javac -d . Simple.java

To Run: java mypack.Simple

```
Output:Welcome to package
```

Access package from another package

There are three ways to access the package from outside the package.

1. import package.*;
2. import package.classname;
3. fully qualified name.

Using packagename.*

- If you use package**.*** then all the classes and interfaces of this package will be accessible but not sub packages.
- The **import** keyword is used to make the classes and interface of another package accessible to the current package.

Example

```
//save by A.java
package pack;
public class A{
  public void msg(){System.out.println("Hello");}
}



//save by B.java
package mypack;
import pack.*;

class B{
 public static void main(String args[]){
 A obj = new A();
 obj.msg();
 }
}
```

```
Output:Hello
```

Subpackage in java

- Package inside the package is called the **subpackage**.
- It should be created **to categorize the package further**.

Example of Subpackage

```
package com.javatpoint.core;
class Simple{
  public static void main(String args[]){
   System.out.println("Hello subpackage");
  }
}
```

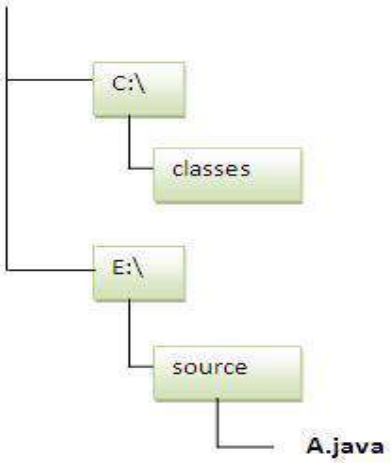
To Compile: javac -d . Simple.java

To Run: java com.javatpoint.core.Simple

```
Output:Hello subpackage
```

Send the class file to another directory or drive

There is a scenario, I want to put the class file of A.java source file in classes folder of c: drive. For example:



```
//save as Simple.java
package mypack;
public class Simple{
 public static void main(String args[]){
    System.out.println("Welcome to package");
   }
}
```

To Compile:

e:\sources> javac -d c:\classes Simple.java

To Run:

To run this program from e:\source directory, you need to set classpath of the directory where the class file resides.

e:\sources> set classpath=c:\classes;.;

e:\sources> java mypack.Simple

```
Output:Welcome to package
```

INTERFACE

An interfaces are declared using **interface** keyword. The variable inside interface are by default static and final.

Some **of the key point of interface as follows:**

- With the help of Interface fully abstraction is achieved in Java.
- In an interface, only abstract method can be declared.
- Interface is used to support multiple inheritance.
- All variable declared inside interface is by default final.
- All method inside interface is by default public and abstract. If you do not define method as public or abstract compiler implicitly treated as public abstract.
- An interface can extends multiple interface.
- An interface is only implemented by a class not extended by a class.
- Method inside interface is ended with semicolon, because method implementation is not allowed only declaration.
- An interface doesn't contain any constructor.
- An interface is implicitly abstract you do not need to use abstract keyword when declaring an interface.

Eg 1:-

```
import java.io.*;
interface Exam1
{
        public static final int m1=100;
        public void average();
}
interface Exam2
{
        public static final int m2=98;
        public void average();
}

class Student implements Exam1,Exam2
{
        public void average()
        {
                int avg=(m1+m2)/2;
                System.out.println("AVERAGE....:"+avg);
        }
}
```

```
class Main
{
        public static void main(String args[])
        {
        Student s=new Student();
        s.average();
        }
}
```

OUTPUT:

AVERAGE….:99

Eg 2:-

```
interface Sastha
{
        public void display1();
        public void display2();
}

class College implements Sastha
{
        public void display1()
        {
                System.out.println("WELCOME TO SSIET");
        }
        public void display2()
        {
                System.out.println("WELCOME TO SSCE");
        }
}
class Inter
{
        public static void main(String args[])
        {
                College c=new College();
                c.display1();
                c.display2();
        }
}
```

OUTPUT:

WELCOME TO SSIET
WELCOME TO SSCE

ABSTRACT CLASS

- An abstract class is a class that is declared by using the **abstract** keyword
- It may or may not have abstract methods. Abstract classes cannot be instantiated, but they can be extended into sub-classes
- An abstract class can be extended into sub-classes, these sub-classes usually provide implementations for all of the abstract methods.
- The key idea with an abstract class is useful when there is common functionality that's like to implement in a superclass and some behavior is unique to specific classes.
- So you implement the superclass as an abstract class and define methods that have common subclasses.
- Then you implement each subclass by extending the abstract class and add the methods unique to the class.
- **Points of abstract class :**

    1. Abstract class contains abstract methods.
    2. Program can't instantiate an abstract class.
    3. Abstract classes contain mixture of non-abstract and abstract methods.
    4. If any class contains abstract methods then it must implements all the abstract methods of the abstract class.

Example:-

```
import java.io.*;
abstract class Abstract
{       public abstract void callme();
        public void display()
        {
        System.out.println("BASE CLASS");
        }
}
class A extends Abstract
{
        public void callme()
        {
        System.out.println("CLASS A");
        }
}
class B extends Abstract
{
        public void callme()
        {
        System.out.println("CLASS B");
        }
}
```

```
class Absdemo
{
        public static void main(String args[])
        {
        Abstract obj1=new A();
        Abstract obj2=new B();
        obj1.callme();
        obj2.callme();
        }
}
```

OUTPUT:-

```
CLASS A
CLASS B
```

Abstract class	Interface
1) Abstract class can **have abstract and non-abstract** methods.	Interface can have **only abstract** methods. Since Java 8, it can have **default and static methods** also.
2) Abstract class **doesn't support multiple inheritance**.	Interface **supports multiple inheritance**.
3) Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
4) Abstract class **can provide the implementation of interface**.	Interface **can't provide the implementation of abstract class**.
5) The **abstract keyword** is used to declare abstract class.	The **interface keyword** is used to declare interface.

6) **Example:**
public abstract class Shape{
public abstract void draw();
}
	Example:
	public interface Drawable{
	void draw();
	}

Internationalization

- **Internationalization** is also abbreviated as I18N because there are total 18 characters between the first letter 'I' and the last letter 'N'.
- Internationalization is a mechanism to create such an application that can be adapted to different languages and regions.

ResourceBundle class in Java

- The **ResourceBundle class** is used to internationalize the messages. In other words, we can say that it provides a mechanism to globalize the messages.

Commonly used methods of ResourceBundle class

- **public static ResourceBundle getBundle(String basename)** returns the instance of the ResourceBundle class for the default locale.
- **public static ResourceBundle getBundle(String basename, Locale locale)** returns the instance of the ResourceBundle class for the specified locale.
- **public String getString(String key)** returns the value for the corresponding key from this resource bundle.

Example of ResourceBundle class

- **MessageBundle_en_US.properties** file contains the localize message for US country.
- **MessageBundle_in_ID.properties** file contains the localize message for Indonaisa country.
- **InternationalizationDemo.java** file that loads these properties file in a bundle and prints the messages.

MessageBundle_en_US.properties

```
greeting=Hello, how are you?
```

MessageBundle_in_ID.properties

```
greeting=Halo, apa kabar?
```

InternationalizationDemo.java

```
import java.util.Locale;
import java.util.ResourceBundle;
public class InternationalizationDemo {
 public static void main(String[] args) {

  ResourceBundle bundle = ResourceBundle.getBundle("MessageBundle", Locale.US);
  System.out.println("Message in "+Locale.US +":"+bundle.getString("greeting"));

  //changing the default locale to indonasian
  Locale.setDefault(new Locale("in", "ID"));
  bundle = ResourceBundle.getBundle("MessageBundle");
  System.out.println("Message in "+Locale.getDefault()+":"+bundle.getString("greeting"
));

 }
}
```

```
Output:Message in en_US : Hello, how r u?
       Message in in_ID : halo, apa kabar?
```

Control Structures

Selection Statement

- Java supports two selection statements: if and switch.

  if statement

  if (condition) statement1**;**

  else statement2**;**

- Each statement may be a single statement or a compound statement enclosed in curly braces (block).
- The condition is any expression that returns a Boolean value.
- The else clause is optional.
- If the condition is true, then statement1 is executed. Otherwise,
- statement2 (if it exists) is executed.
- In no case will both statements be executed.

Nested ifs

- A nested if is an if statement that is the target of another if or else.
- In nested ifs an else statement always refers to the nearest if statement that is within the same block as the else and that is not already associated with an else.

```
if (i == 10) {
if (j < 20) a = b;
if (k > 100) c = d; // this if is
else a = c; // associated with this else
}
else a = d; // this else refers to if(i == 10)
```

The if-else-if Ladder

- A sequence of nested ifs is the if-else-if ladder.

```
if(condition)
statement;
else if(condition)
statement;
else if(condition)
statement;
...
else
statement;
```

- The if statements are executed from the top to down.

switch

- The switch statement is Java's multi-way branch statement.
- provides an easy way to dispatch execution to different parts of your code based on the value of an expression.
- provides a better alternative than a large series of if-else-if statements.

```
switch (expression) {
case value1:
// statement sequence
break;
case value2:
// statement sequence
break;
...
case valueN:
// statement sequence
break;
default:
// default statement sequence
}
```

- The expression must be of type byte, short, int, or char.
- Each of the values specified in the case statements must be of a type compatible with the expression.
- Each case value must be a unique literal (i.e. constant not variable).
- Duplicate case values are not allowed.
- The value of the expression is compared with each of the literal values in the case statements.
- If a match is found, the code sequence following that case statement is executed.
- If none of the constants matches the value of the expression, then the default statement is executed.
- The default statement is optional.
- If no case matches and no default is present, then no further action is taken.
- The break statement is used inside the switch to terminate a statement sequence.
- When a break statement is encountered, execution branches to the first line of code that follows the entire switch statement.

```
class SampleSwitch {
public static void main(String args[]) {
for(int i=0; i<6; i++)
switch(i) {
case 0:
System.out.println("i is zero.");
break;
```

```
case 1:
System.out.println("i is one.");
break;
case 2:
System.out.println("i is two.");
break;
default:
System.out.println("i is greater than 2.");
}
}
}
```

Nested switch Statements

- When a switch is used as a part of the statement sequence of an outer switch. This is called a nested switch.

```
switch(count) {
case 1:
switch(target) { // nested switch
case 0:
System.out.println("target is zero");
break;
case 1: // no conflicts with outer switch
System.out.println("target is one");
break;
}
break;
case 2: // ...
```

Difference between ifs and switch

- switch can only test for equality, whereas if can evaluate any type of Boolean expression. That is, the switch looks only for a match between the value of the expression and one of its case constants.
- A switch statement is usually more efficient than a set of nested ifs.
- Note: No two case constants in the same switch can have identical values. Of course, a switch statement and an enclosing outer switch can have case constants in common.

IteratIon Statement
(Loops)

Iteration Statements

- In Java, iteration statements (loops) are:
  - for
  - while, and
  - do-while
- A loop repeatedly **executes the same set of instructions until a termination** condition is met.

While Loop

- While loop repeats a statement or block while its controlling expression is true.
- The condition can be any Boolean expression.
- The body of the loop will be executed as long as the conditional expression is true.
- When condition becomes false, control passes to the next line of code immediately following the loop.

```
while(condition)
{
// body of loop
}
```

Eg:

```
class While
{
public static void main(String args[]) {
int n = 10;
char a = 'G';
        while(n > 0)
        {
                System.out.print(a);
                n--;
                a++;
        }
}
}
```

- The body of the loop will not execute even once if the condition is false.
- The body of the while (or any other of Java's loops) can be empty. This is because a null statement (one that consists only of a semicolon) is syntactically valid in Java.

do-while

- The do-while loop always executes its body at least once, because its conditional expression is at the bottom of the

loop.

```
do {
// body of loop
} while (condition);
```

- Each iteration of the do-while loop first executes the body of the loop and then evaluates the conditional expression.
- If this expression is true, the loop will repeat. Otherwise, the loop terminates.

for Loop

```
for (initialization; condition; iteration)
{
// body
}
```

- Initialization portion sets the value of loop control variable.
  Initialization expression is only executed once.
- Condition must be a Boolean expression. It usually tests the loop control variable against a target value.
- Iteration is an expression that increments or decrements the loop control variable.

The for loop operates as follows.

- When the loop first starts, the initialization portion of the loop is executed.
- Next, condition is evaluated. If this expression is true, then the body of the loop is executed. If it is false, the loop terminates.
- Next, the iteration portion of the loop is executed.

```
class ForTable
{
public static void main(String args[])
{
int n;
int x=5;
for(n=1; n<=10; n++)
{
int p = x*n;
System.out.println(x+"*"+n +"="+ p);
}
}
}
```

For-Each Version of the for Loop

- Beginning with JDK 5, a second form of for was defined that implements a "for-each" style loop.
- For-each is also referred to as the **enhanced for loop.**
- Designed to cycle through a collection of objects, such as an array, in strictly sequential fashion, from start to end.

for (type itr-var : collection) statement-block

- **type** specifies the type.
- **itr-var** specifies the name of an iteration variable that will receive the elements from a collection, one at a time, from beginning to end.
- The collection being cycled through is specified by collection.
- With each iteration of the loop, the next element in the collection is retrieved and stored in itr-var.
- The loop repeats until all elements in the collection have been obtained.

```
class ForEach
{
public static void main(String arr[])
{
int num[] = { 1, 2, 3, 4, 5 };
int sum = 0;
    for(int i : num)
    {
    System.out.println("Value is: " + i);
    sum += i;
    }
System.out.println("Sum is: " + sum);
}
}
```

return

- The return statement is used to explicitly return from a method.
- It causes program control to transfer back to the caller of the method.
- Example:

```
class Return {
public static void main(String args[]) {
boolean t = true;
System.out.println("Before the return.");
if(t) return; // return to caller
System.out.println("This won't execute.");
}
}
```